

第十一章 一天面试速记（面试前30分钟必读★★★★★）

阅读指南： 这是你面试前30分钟过一遍的救命文档。超级浓缩，全是干货。每个知识点一句话或一张表。全文档约1000行，20页打印内容。

第一部分：C++核心速记（2页）

1.1 左值/右值/移动语义 一句话

- **左值：** 有名字、可寻址（变量、函数返回左值引用）
- **右值：** 匿名、临时（字面量、函数返回值、std::move(x)的结果）
- **右值引用(T&&)：** 绑定右值，用于移动语义
- **std::move：** 无条件转右值。语义"我不再需要了"
- **std::forward：** 有条件转发（完美转发），左值进左值出、右值进右值出
- **移动后对象：** valid but unspecified（可析构、可赋值，值不保证）
- **万能引用：** T&& + 模板推导才生效，const T&&不是

1.2 智能指针选择速查表

智能指针	所有权	何时用	禁止
unique_ptr	独占	默认首选，Pimpl，工厂函数返回	不可拷贝
shared_ptr	共享	多个所有者，图结构	循环引用(用weak_ptr解)
weak_ptr	无(观察)	配合shared_ptr，打破循环引用	直接使用(需lock())
auto_ptr	—	已废弃，不许用	C++11前遗留

- **make_shared vs new shared_ptr：** make一次分配，但weak_ptr多时内存延迟释放；new两次分配但控制更灵活
- **循环引用：** 两个shared_ptr互持→永不释放。一方改用weak_ptr
- **enable_shared_from_this：** 成员函数返回自身shared_ptr时用，避免double free
- **Rule of thumb：** 能用unique_ptr就别用shared_ptr；堆上对象必有管理者

1.3 虚函数表结构简图

对象内存布局（64位）：

+-----+ +-----+

| vptr(8B) |---->| type_info | RTTI信息

+-----+ |-----+

| members | | &func1() | 虚函数表(rodata段)

| ... | | &func2() |

+-----+ | ... |

+-----+

单继承：1张vtable，派生类override替换对应槽，新虚函数追加

多重继承：N个基类→N张vtable（每个基类子对象一张）

虚继承：vbtable(虚基类表)，间接偏移量

1.4 模板类型推导 一句话口诀

万能引用：实参为左值 → T = T& → 结果T&

实参为右值 → T = T → 结果T&&

auto：跟模板推导一样（auto&&也是万能引用）

decltype(auto)：原封不动保留引用+const

1.5 Lambda捕获方式速查

捕获方式	含义
[]	无捕获（纯函数指针可转换）
[=]	值捕获全部（this隐式捕获）
[&]	引用捕获全部
[x]	值捕获x
[&x]	引用捕获x
[=, &x]	默认值捕获，x引用捕获
[*this] (C++17)	捕获this的拷贝

1.6 constexpr vs const

关键词	含义	何时求值
const	不可修改	运行时/编译期
constexpr	编译期常量	必须编译期
constexpr函数	可能编译期执行	参数为constexpr时编译期

1.7 其他核心速记

- **explicit**: 禁止隐式类型转换（单参构造函数+转换运算符）
- **noexcept**: 不抛异常保证；移动构造必须加（否则vector扩容退回拷贝）
- **=default**: 要求编译器生成默认特殊成员
- **=delete**: 禁止函数使用（如禁止拷贝）
- **override**: 编译器检查真的覆盖了基类虚函数
- **final**: 虚函数不能继续被覆盖/类不能被继承
- **RAII**: 资源获取即初始化=构造获取，析构释放。C++最核心范式
- **Rule of Five**: 定义了析构/拷贝/移动中任一，定义全部5个
- **Rule of Zero**: 资源由智能指针等RAII管理，不自定义任何特殊成员

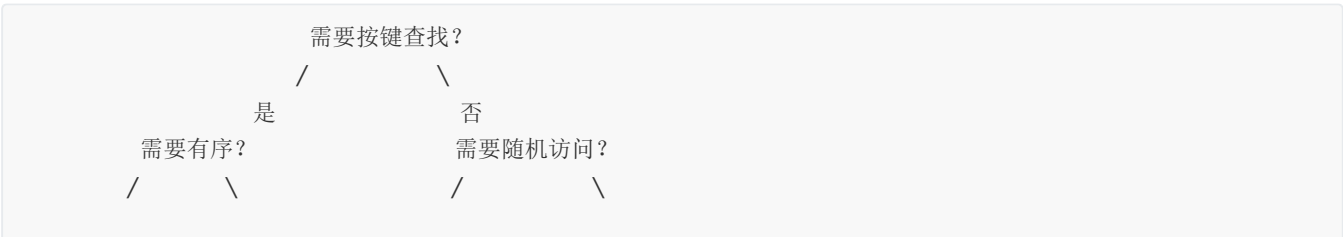
第二部分：STL容器速查表（2页）

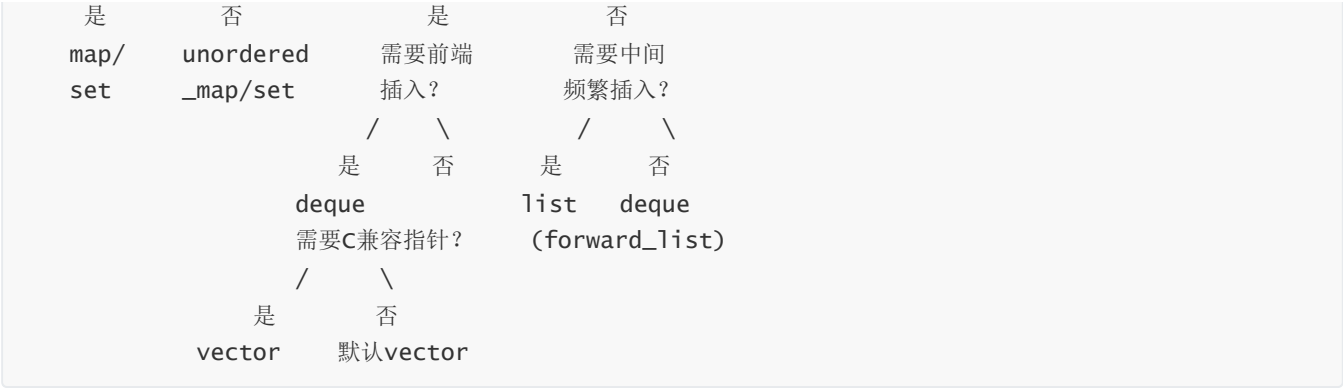
2.1 所有容器时间复杂度

容器	随机访问	插入(前)	插入(后)	插入(中)	查找	内存
vector	O(1)	—	O(1)均	O(n)	O(n)	连续
deque	O(1)	O(1)	O(1)	O(n)	O(n)	分段连续
list	—	O(1)	O(1)	O(1)	O(n)	双向链表
forward_list	—	O(1)	—	O(1)	O(n)	单向链表
map/set	—	O(log n)	O(log n)	O(log n)	O(log n)	红黑树
unordered_map/set	—	O(1)均	O(1)均	O(1)均	O(1)均	哈希表
stack/queue	—	—	O(1)	—	—	deque适配
priority_queue	—	—	O(log n)	—	O(1)顶	堆(vector)
array	O(1)	—	—	—	O(n)	连续
string	O(1)	—	O(1)均	O(n)	O(n)	连续+SSO

关键数字： vector 扩容 1.5x(GCC) 或 2x(MSVC)。deque chunk 大小 512B 或 sizeof(T)。unordered_map 默认桶数通常质数。

2.2 容器选择决策树





2.3 迭代器失效速查

容器	push/insert	erase	重新分配
vector	可能全部失效	位置及之后失效	全部失效
deque	可能全部失效	全部失效	—
list	不失效	仅被删失效	—
map/set	不失效	仅被删失效	—
unordered	若rehash全部失效	仅被删失效	rehash时全部失效

2.4 扩容机制一句话总结

- **vector**: 1.5x(GCC,复用旧内存可能) / 2x(MSVC)。realloc新内存+移动/拷贝+释放旧
- **string SSO**: 短字符串(<15~22字节)栈上存, 超过阈值堆分配
- **unordered_map**: 负载因子>max_load_factor()时rehash(所有桶重新分配+元素重新哈希放入)
- **deque**: 新开chunk, 不需要拷贝所有元素 (仅新增chunk)

第三部分：Linux必背 (2页)

3.1 进程 vs 线程对比表

维度	进程	线程
资源	独立地址空间	共享地址空间
创建	fork() COW	pthread_create()
通信	IPC(管道/共享内存/消息队列等)	直接读写共享变量(需同步)
切换	重(TLB flush等)	轻(寄存器切换)
独立	一崩溃不影响其他	一崩溃可能全进程崩溃
内核结构	task_struct(不同mm)	task_struct(共享mm)

- **fork()**: COW, 父子共享物理页直到任一写操作。fork后fd共享(引用计数+1)
- **孤儿进程**: 父先亡, init收养, 无危害
- **僵尸进程**: 子亡父不wait, PCB不释放。解决: wait/signal(SIGCHLD, SIG_IGN)/双fork

3.2 内存管理流程图 (ASCII)



3.3 IPC方式对比表

IPC	速度	数据量	有无血缘	同步
共享内存	最快	大	任意	需手动
mmap	很快	大	任意	需手动
消息队列	较快	中	任意	自带
管道/FIFO	中等	小	pipe需/可任意	流式
信号量	—	原子操作	任意	自带
信号	慢	极少	任意	异步
套接字	中等~慢	大	跨网络	TCP自带

3.4 malloc原理一句话

- **小内存(<128KB)**: brk()移动堆顶。空闲链表管理。free()不一定归还OS (内部碎片管理)
- **大内存(>=128KB)**: mmap()匿名映射。free()立即munmap()归还
- **free如何知大小**: ptr前的chunk header存size字段。free(ptr) → *(ptr-header_offset) 取size
- **ptmalloc竞技**: 多线程malloc/free竞争→arena机制分摊锁 (类似分区锁)

3.5 系统调用流程

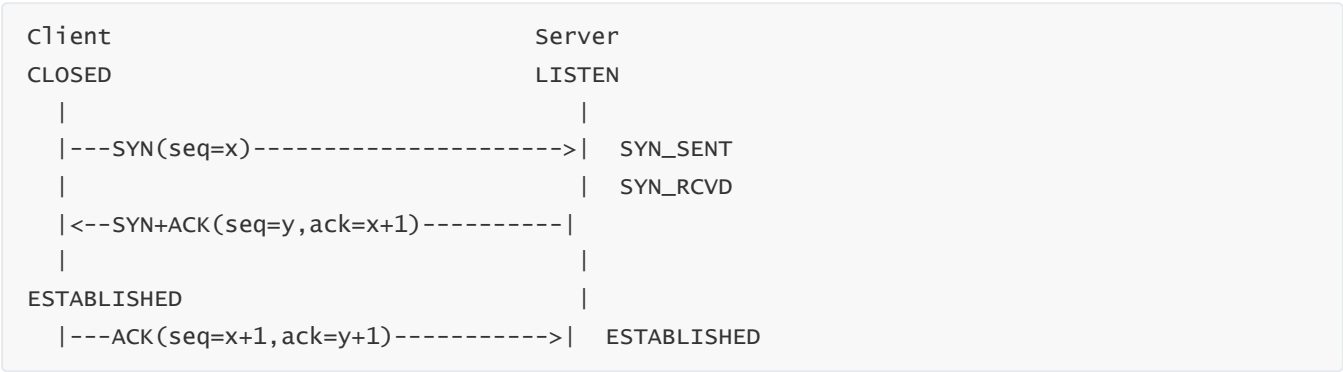


3.6 其他关键知识点

- **零拷贝**: sendfile/splice, DMA在磁盘<->网卡直通, CPU不做数据拷贝
- **缺页中断**: 页不在物理内存。3种: minor(已在内存, 只更新页表)、major(需磁盘IO)、invalid(SIGSEGV)
- **TLB**: MMU内虚拟->物理映射缓存。TLB miss→查页表。大页可减少TLB miss
- **伙伴系统**: 物理页框分配器, 2^N 连续页。分裂/合并
- **slab**: 内核对象缓存(伙伴系统之上)。减少频繁从伙伴系统分配
- **CFS调度器**: vruntime公平调度, 红黑树选最小值。nice值映射为权重

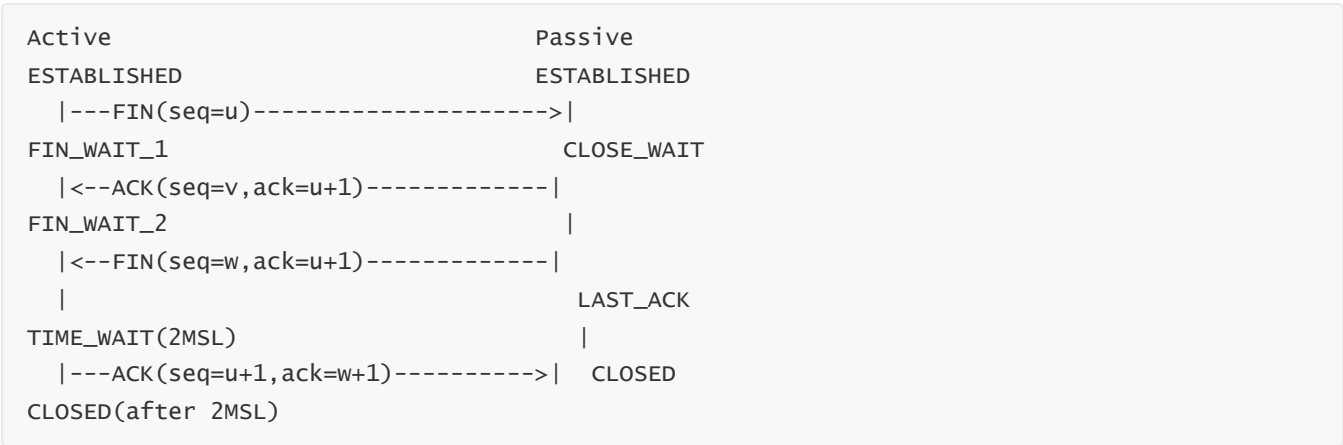
第四部分：网络必背（3页）

4.1 三次握手状态变迁图



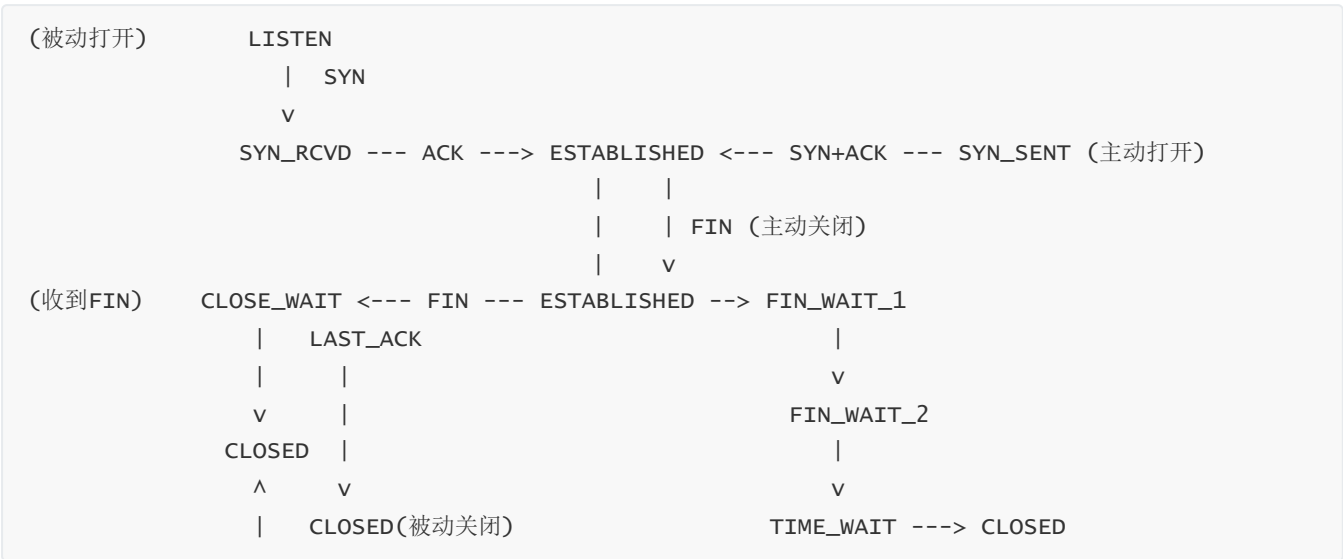
为什么三次? 两次Server不能确认Client收能力, 四次多余的包。三握保证双方各确认对方收发正常。

4.2 四次挥手状态变迁图



TIME_WAIT为什么2MSL? 1)确保最后的ACK可被重传 2)让旧连接报文在网络中消失。Linux默认60s(2MSL=120s)

4.3 TCP状态机简图



4.4 select/poll/epoll 对比表（最重要的表！）

维度	select	poll	epoll
数据结构	fd_set位图	pollfd数组	红黑树+就绪链表
最大fd数	1024(默认)	无限制	无限制
用户态→内核态	每次全量拷贝fd_set	每次全量拷贝数组	仅epoll_ctl时拷贝+mmap共享
就绪fd查找	O(n)遍历	O(n)遍历	O(1)从就绪链表取
水平触发	是	是	是(默认)
边缘触发	否	否	是
适用规模	<1K连接	<1K连接	任意规模

epoll快在哪? 事件驱动回调注册(红黑树)+就绪链表+mmap共享内存。不遍历不拷贝。

4.5 ET(边缘触发) vs LT(水平触发) 速查

特性	LT	ET
通知方式	有数据就一直通知	状态改变时通知一次
读方式	可以部分读	必须读到EAGAIN
阻塞I/O	可配合	必须非阻塞
编程难度	简单	复杂
性能	可能重复通知浪费	高并发更高效

ET关键： 1)必须非阻塞socket 2)必须循环read/write到EAGAIN 3)丢事件=数据留在缓冲区不被读(状态不再变)

4.6 拥塞控制四算法

事件驱动状态转移：

1. 慢启动：cwnd=1，每ACK cwnd+1 → 指数增长(RTT翻倍)。条件：cwnd < ssthresh

2. 拥塞避免：每RTT cwnd+1 → 线性增长。条件：cwnd >= ssthresh

3. 快速重传：收到3个重复ACK → 立即重传(不等超时)

4. 快速恢复：{ssthresh=cwnd/2；cwnd=ssthresh+3}。跳过慢启动继续拥塞避免(TCP Reno)

现代常用算法：CUBIC(Linux默认，三次函数)、BBR(Google，基于带宽/延迟估计)

4.7 HTTP状态码速查

2xx成功： 200 OK | 201 Created | 204 No Content | 206 Partial Content

3xx重定向： 301 永久 | 302 临时 | 304 Not Modified(缓存) | 307/308 不改变方法

4xx客户端： 400 Bad Request | 401 Unauthorized | 403 Forbidden | 404 Not Found | 405 Method | 429 Rate Limit

5xx服务端： 500 Internal Error | 502 Bad Gateway | 503 Unavailable | 504 Gateway Timeout

4.8 HTTP版本对比表

版本	连接	多路复用	头部压缩	服务器推送	队头阻塞
HTTP/1.1	Keep-Alive	否(管道化几乎不可用)	否	否	有
HTTP/2	持久连接	是(多流)	HPACK	Server Push	应用层解决(流层仍有TCP层队头阻塞)
HTTP/3	QUIC(UDP)	是	QPACK	是	彻底解决(各流独立)

4.9 网络编程其他关键点

- **Reactor vs Proactor**: Reactor="数据就绪通知你，你自己读"(epoll+read); Proactor="我帮你读好了，你直接用结果"(io_uring+IOCP)
- **惊群**: 多线程/进程同时wait同一fd，全部被唤醒但只有一个成功。Linux 3.9+解决accept惊群，epoll仍有 (EPOLLEXCLUSIVE/REUSEPORT缓解)
- **粘包/拆包解决方案**: 消息头(固定长度字段+体) | 分隔符 | 定长消息
- **SYN flood**: 大量SYN不完成握手→SYN队列满。防御: SYN cookies、增加backlog
- **大量TIME_WAIT**: 服务器主动关闭太多。解决: 连接池/keepalive、tcp_tw_reuse、让客户端主动关闭
- **CLOSE_WAIT过多**: 应用层bug——收到FIN后没close()。检查recv=0的处理逻辑
- **Nagle + 延迟ACK互相拖累**→灾难性延迟。游戏用TCP_NODELAY关闭Nagle

第五部分：多线程必背（2页）

5.1 互斥锁类型对比表

锁类型	适用场景	注意
std::mutex	一般互斥	不可递归(重复lock死锁)
std::recursive_mutex	同一线程可多次加锁	每次需等量unlock
std::shared_mutex(C++17)	读多写少	shared_lock(读) + unique_lock(写)
std::timed_mutex	带超时的try_lock	try_lock_for/try_lock_until
std::atomic_flag	自旋锁实现	最轻量

RAII锁包装: lock_guard(简单)、unique_lock(可延迟/转移/配合条件变量)、scoped_lock(C++17,同时锁多个mutex防死锁)

5.2 memory order速查表

memory_order	用途	关键
relaxed	简单计数、不关心顺序	仅原子性，无顺序保证
acquire	load时获取	后续读/写不重排到此之前
release	store时释放	之前读/写不重排到此之后
acq_rel	RMW操作(如fetch_add)	acquire+release
seq_cst	默认，顺序一致性	全局统一顺序，最安全最慢

关键组合: A store(release) → B load(acquire) → B能看到A在store前的所有写操作

5.3 死锁四条件

互斥 + 持有并等待 + 不可抢占 + 循环等待 → 死锁

|

破坏任一条件=预防

预防最快记法：一次性获取所有锁(std::scoped_lock) 或 统一锁顺序(按地址排序)

5.4 线程池核心代码骨架

```
class ThreadPool {
    queue<function<void()>> tasks; mutex m; condition_variable cv;
    vector<thread> workers; bool stop;

    void worker() {
        while (true) {
            function<void()> task;
            { unique_lock lk(m); cv.wait(lk, [&]{ return stop || !tasks.empty(); });
              if (stop && tasks.empty()) return;
              task = move(tasks.front()); tasks.pop(); }
            task();
        }
    }
    void enqueue(F f) { { lock_guard lk(m); tasks.push(f); } cv.notify_one(); }
    ~ThreadPool() { { lock_guard lk(m); stop = true; } cv.notify_all();
                    for (auto& t : workers) if (t.joinable()) t.join(); }
};
```

5.5 atomic vs mutex 场景选择

场景	选择	原因
简单计数器	atomic	轻量，无锁
多变量原子修改	mutex	atomic不能保证多变量间原子性
复杂数据结构	mutex	atomic范围有限
等条件满足	condition_variable+mutex	atomic无wait能力
极短临界区(几指令)	atomic(spinlock)	无上下文切换
中等/长临界区	mutex	阻塞不浪费CPU

5.6 其他多线程关键点

- **volatile != 线程安全**：C++ volatile 仅禁优化(每次从内存读)，不插内存屏障、不保证原子。线程间用atomic
- **虚假唤醒**：condition_variable 的 wait 必须用 while 检查条件(或 wait 的 lambda 形式)
- **伪共享**：两个变量同 cache line(64B) → 各线程改各变量也互相拖累。解决：padding(aligned to cache line)
- **ABA问题**：lock-free CAS中 A被改成B又改回A。解决：加版本号(双宽CAS)

- **DCLP问题**: C++11前双重检查锁单例不安全(构造/赋值重排)。C++11后用Meyers Singleton(static局部变量)
- **thread_local**: 每线程独立副本, 通过FS/GS段寄存器访问
- **call_once**: 高效的一次性初始化, 内部的futex实现
- **future/promise**: 单向值传递通道(promise设值, future取结果)
- **fork多线程陷阱**: fork只复制调fork的线程→其他线程锁永远不释。应fork后立即exec

第六部分：设计模式速查（1页）

6.1 七种模式一句话+适用场景

模式	一句话	何时用
单例	全局唯一实例	配置/日志/连接池
工厂方法	创建接口, 子类决定具体产品	多态创建, 扩展不需改代码
抽象工厂	创建相关对象族	UI风格 (一套控件家族)
策略	封装算法族, 可互换	排序器、支付方式选择
观察者	一对多通知	事件驱动、GUI、MQ订阅
适配器	转接口	遗留代码兼容、STL适配器(stack等)
装饰器	动态加功能	给Stream加压缩/加密
建造者	分步创建复杂对象	多配置构建
模板方法	父定义骨架, 子实现步骤	框架的生命周期
状态	状态驱动行为变化	TCP状态机、订单状态流转

6.2 SOLID 一句话速记

原则	一句话
S 单一职责	一个类只有一个变化原因
O 开闭	对扩展开放(新增类), 对修改关闭(不改已有代码)
L 里氏替换	派生类必须能完全替代基类
I 接口隔离	多专用接口优于一个胖接口
D 依赖倒转	依赖抽象(接口), 不依赖具体实现

6.3 C++设计要点

- **Meyers Singleton** = 函数内static局部变量（C++11线程安全初始化）— 最简单最安全
- **组合优先于继承**：HAS-A > IS-A。用组合获得灵活性
- **RAII**：C++独有设计范式，资源生命周期绑定对象生命周期
- **Pimpl**：隐藏实现细节，编译防火墙，ABI稳定
- **CRTP**：编译期多态，零开销（vs 虚函数运行时开销）

第七部分：面试高频问题TOP30（3页）

按热度排序，每题3-4句精炼答案

TOP 1-10

1. 多态与虚函数表？

运行时多态通过vtable实现：类有vtable指针vp_ptr→对象前8B置vp_ptr→派生类override替换对应槽。构造/析构中虚函数不触发多态（vp_ptr指向当前类）。多重继承每基类一张vtable。

2. TCP三次握手和四次挥手？

三握：SYN→SYN+ACK→ACK，确保双方收发能力。四挥：FIN→ACK→FIN→ACK，主动关闭方进入TIME_WAIT等2MSL。三握非二因需确认彼此收发，非四因ACK+SYN可合并。

3. 进程和线程区别？

进程是资源分配单位（独立地址空间），线程是CPU调度单位（共享地址空间）。Linux都是task_struct，共享mm为线程。线程切换开销小（不刷TLB），通信直接、需同步。

4. malloc和new的区别？

new是运算符（调构造+分配），malloc是函数（只分配）。new返回正确类型、失败抛异常、不可与free混用。new = operator new + placement new。

5. select/poll/epoll区别？

epoll快在事件驱动+红黑树+就绪链表+mmap共享内存，O(1)取就绪。select/poll每次全量拷贝+O(n)遍历。epoll支持ET(高效)+LT(简单)。Redis/Nginx都用epoll。

6. 虚析构函数为什么需要？

基类指针删派生对象→基类析构非虚→只调基类析构→派生类部分泄漏。可能被继承的基类应声明虚析构。不需要多态的类不需要（加vp_ptr开销8B）。

7. 智能指针的选择与循环引用？

unique_ptr独占优先。shared_ptr共享但可能循环引用（互相持有→永不释放→用weak_ptr破坏）。make_shared一次分配好但weak_ptr多时延迟释放。

8. vector扩容机制？

底层连续数组，size==capacity时扩容：重新分配1.5x(GCC)或2x(MSVC)新内存→移动/拷贝元素→释放旧内存。均摊O(1)。扩容使迭代器/指针/引用失效。

9. 死锁的四个条件与预防？

互斥+持有并等待+不可抢占+循环等待=死锁。破坏循环等待：统一锁顺序或scoped_lock同时获取。破坏持有等待：一次性申请所有锁。

10. 左值和右值、move和forward?

左值可寻址（变量），右值临时（字面量/临时对象）。move无条件转右值，forward有条件转发（完美转发）。move后对象valid但unspecified。万能引用=T&&+模板推导。

TOP 11-20

11. map和unordered_map区别?

map红黑树 $O(\log n)$ 、有序、迭代器稳定。unordered_map哈希表 $O(1)$ 均摊、无序、rehash时迭代器失效。需要有序遍历选map；纯查找选unordered_map。

12. RAII是什么?

资源获取即初始化：构造获取，析构释放。利用C++确定性析构保证异常安全。典型：lock_guard(锁)、unique_ptr(内存)、ifstream(文件)。C++最核心的编程范式。

13. 移动语义解决什么?

避免不必要深拷贝。vector移动只需交换3个指针($O(1)$)，拷贝需分配+逐元素复制($O(n)$)。通过右值引用区分"可安全劫持"的临时对象。移动构造须noexcept。

14. static的各种用法?

static局部变量(生命周期整个程序，C++11线程安全)；static全局(限制当前编译单元，防命名冲突)；static成员变量(类内共享，类外定义)；static成员函数(无this，只访问static成员)。

15. const的各种用法?

const变量(不可改)；const指针(三种：指向const/指针const/都const)；const成员函数(不可改成员，mutable例外)；const参数(const T&防修改零拷贝)；顶层vs底层const。

16. 四次挥手TIME_WAIT?

主动关闭方进入TIME_WAIT等2MSL(Linux 60s→120s)。原因：1)确保最后ACK可重传 2)让旧连接报文在网络消失。大量TIME_WAIT可调tcp_tw_reuse；让客户端主动关闭。

17. epoll ET/LT区别?

LT有数据一直通知(简单安全)，ET状态变化时通知一次(高效)。ET必须非阻塞+循环读到EAGAIN。ET高并发更优，LT编程简单。Nginx用ET，Redis用LT。

18. 构造函数能否虚？析构函数呢?

构造不能虚(vptr在构造中设置，构造前vptr无效→无法虚查找)。析构可以且常常应该虚(基类指针删派生对象)。C++标准明确规定构造不能virtual。

19. 原子操作与CAS?

原子操作不可中断。CAS(Compare-And-Swap)：比较并交换，无锁编程基本原语。C++用compare_exchange_weak/strong。ABA问题需版本号。memory_order控制可见性顺序。

20. 线程同步方式?

mutex(一般互斥)、shared_mutex(读多写少)、condition_variable(等条件)、atomic(简单计数)、spinlock(极短临界区)、semaphore(控制并发数C++20)、barrier/latch(C++20)。

TOP 21-30

21. TCP如何保证可靠?

序列号+确认、超时重传、滑动窗口(流量控制)、拥塞控制(慢启动/拥塞避免/快速重传/快速恢复)、校验和。TCP保证可靠有序字节流(但无消息边界→粘包)。

22. 类型转换四种cast？

static_cast(相关类型,编译期)、dynamic_cast(多态下行,运行时检查)、const_cast(+-const)、reinterpret_cast(比特重解释,最危险)。dynamic_cast需RTTI，失败返回nullptr。

23. C++11/14/17新特性？

C++11:auto/右值引用/智能指针/Lambda/nullptr。C++14:泛型Lambda/make_unique。C++17:结构化绑定/if constexpr/optional/CTAD。

24. 深拷贝和浅拷贝？

浅拷贝逐字节复制→指针成员共享→double free。深拷贝分配独立内存复制内容。有指针成员须定义拷贝/赋值/析构(Rule of Three)。

25. DNS解析过程？

浏览器缓存→OS缓存→hosts→本地DNS→根DNS→顶级域DNS→权威DNS→返回IP。基于UDP(53端口)，大响应用TCP fallback。

26. TCP粘包拆包？

TCP无消息边界。解决：固定消息头(长度字段+体)、特殊分隔符、定长消息。应用层接收缓冲+循环解析。

27. 内存泄漏排查？

Valgrind(运行时动态检查,10-20x慢)→AddressSanitizer(编译期插桩,~2x开销)。ASan用影子内存和红区检测溢出和use-after-free。

28. 单例线程安全实现？

C++11 Meyers Singleton：函数内static局部变量自动线程安全初始化。安全、简洁、高效，优于双重检查锁。编译器插入__cxa_guard_acquire保护。

29. 零拷贝？

避免用户空间与内核空间间的CPU拷贝。sendfile→数据从磁盘DMA到内核再到网卡不需到用户空间。splice在fd间移动数据也不经过用户空间。文件服务器场景性能提升显著。

30. HTTP 1.1 vs 2.0？

1.1串行请求，文本协议；2.0多路复用(多流+帧)、HPACK头部压缩、二进制协议、服务器推送。2.0解决应用层队头阻塞但TCP层仍存在(HTTP/3基于QUIC(UDP)彻底解决)。

关键数字速记

数字	含义
2MSL = 60s(Linux)	TIME_WAIT等待时间 (2MSL=120s)
1.5x / 2x	vector扩容因子 (GCC/MSVC)
1024	select最大fd数 (FD_SETSIZE)
8MB	Linux默认线程栈大小
65535	端口号上限 (2^16-1)
3/4	三次握手次数 / 四次挥手次数
64B	CPU Cache Line大小
3次重复ACK	触发TCP快速重传

数字	含义
O(log n)	红黑树操作复杂度 (map/set)
O(1)均摊	哈希表/unordered_map插入查找 (最坏O(n))
7200s	TCP keepalive默认空闲时间 (2小时)
128KB	brk/mmap 分界线 (glibc malloc)
0-2	stdin/stdout/stderr 默认fd

第八部分：手写代码速成（2页）

8.1 单例模式（最简线程安全版）

```
class Singleton {
public:
    static Singleton& get() {
        static Singleton inst; // C++11 Magic Statics
        return inst;
    }
    Singleton(const Singleton&) = delete;
    void operator=(const Singleton&) = delete;
private:
    Singleton() = default;
};
```

8.2 线程池骨架

```
class ThreadPool {
    vector<thread> workers;
    queue<function<void()>> tasks;
    mutex mtx; condition_variable cv; bool stop = false;
public:
    ThreadPool(size_t n) {
        for (size_t i = 0; i < n; ++i)
            workers.emplace_back([this] {
                while (true) {
                    function<void()> task;
                    { unique_lock lk(mtx);
                      cv.wait(lk, [this]{ return stop || !tasks.empty(); });
                      if (stop && tasks.empty()) return;
                      task = move(tasks.front()); tasks.pop(); }
                    task();
                }
            });
    }
    template<class F> void enqueue(F f) {
        { lock_guard lk(mtx); tasks.emplace(f); }
```

```

        cv.notify_one();
    }
    ~ThreadPool() {
        { lock_guard lk(mtx); stop = true; }
        cv.notify_all();
        for (auto& w : workers) if (w.joinable()) w.join();
    }
};

```

8.3 生产者消费者骨架

```

mutex mtx; condition_variable cv;
queue<int> q; const int MAX = 10;

void producer() {
    for (int i = 0; ; ++i) {
        unique_lock lk(mtx);
        cv.wait(lk, []{ return q.size() < MAX; });
        q.push(i); lk.unlock(); cv.notify_one();
    }
}

void consumer() {
    while (true) {
        unique_lock lk(mtx);
        cv.wait(lk, []{ return !q.empty(); });
        int v = q.front(); q.pop(); lk.unlock(); cv.notify_one();
        // process v
    }
}

```

8.4 shared_ptr简化实现

```

template<typename T>
class SharedPtr {
    T* ptr = nullptr;
    size_t* refCount = nullptr;
public:
    explicit SharedPtr(T* p = nullptr) : ptr(p), refCount(new size_t(1)) {}
    SharedPtr(const SharedPtr& other) : ptr(other.ptr), refCount(other.refCount) {
        ++(*refCount);
    }
    ~SharedPtr() {
        if (--(*refCount) == 0) { delete ptr; delete refCount; }
    }
    SharedPtr& operator=(SharedPtr other) { swap(ptr, other.ptr); swap(refCount,
other.refCount); return *this; }
    T* operator->() const { return ptr; }
    T& operator*() const { return *ptr; }
};

```

8.5 string简化实现

```
class String {
    char* data;
    size_t len;
public:
    String(const char* s) : len(strlen(s)), data(new char[len+1]) {
        strcpy(data, s);
    }
    String(const String& other) : len(other.len), data(new char[len+1]) {
        strcpy(data, other.data);
    }
    String& operator=(String other) { swap(data, other.data); swap(len, other.len); return
*this; }
    ~String() { delete[] data; }
    // 移动构造 (可选)
    String(String&& other) noexcept : data(other.data), len(other.len) {
        other.data = nullptr; other.len = 0;
    }
};
```

8.6 vector简化实现

```
template<typename T>
class Vector {
    T* data; size_t sz, cap;
public:
    Vector() : data(nullptr), sz(0), cap(0) {}
    void push_back(const T& val) {
        if (sz == cap) { reserve(cap == 0 ? 1 : cap * 2); }
        data[sz++] = val;
    }
    void reserve(size_t newCap) {
        if (newCap <= cap) return;
        T* newData = static_cast<T*>(::operator new(newCap * sizeof(T)));
        for (size_t i = 0; i < sz; ++i) new (&newData[i]) T(data[i]); // placement new
        for (size_t i = 0; i < sz; ++i) data[i].~T();
        ::operator delete(data);
        data = newData; cap = newCap;
    }
    size_t size() const { return sz; }
    size_t capacity() const { return cap; }
    ~Vector() { for (size_t i = 0; i < sz; ++i) data[i].~T(); ::operator delete(data); }
};
```

第九部分：面试反问环节（1页）

9.1 可以反问面试官的好问题

关于团队和技术：

- "团队目前的技术栈是怎样的？主要用什么语言和框架？"
- "团队现在主要解决什么技术问题或业务挑战？"
- "代码审查(code review)是怎么做的？"
- "技术决策是怎样的——自上而下还是团队共识？"
- "你们用什么工具做CI/CD、测试、部署？"

关于个人成长：

- "这个岗位的新人通常如何 onboarding？有什么培训体系？"
- "技术晋升的路径是怎样的？有技术专家和管理两条轨道吗？"
- "团队鼓励参与开源或技术会议吗？"

关于项目和方向：

- "接下来6个月团队的关键目标是什么？"
- "这个岗位上最大的挑战会是什么？"
- "你们最近遇到的最有趣的技术问题是什么？"

9.2 应该问什么

应该问的	为什么
团队规模/结构	了解工作环境
技术栈/开发流程	评估是否匹配自己的技能
项目当前阶段	了解你需要做什么(从0到1 vs 维护)
对新人的期望	了解考核标准
成长路径	展示长期发展意向

9.3 不应该问什么

不应该问的	为什么
"我表现怎么样？" / "什么时候有结果？"	制造尴尬，面试官会主动通知
工资/福利/加班（过早）	留给HR面，技术面问不专业
"公司做什么的？"	面试前应做好功课
过于琐碎的问题	显得不关心核心工作
不尊重的措辞	"你们技术债务严重吗？" "为什么这么选？"

第十部分：最后检查清单（1页）

10.1 面试前检查清单

- ☐ **简历**：纸质版3份，重点突出，无错别字
- ☐ **着装**：科技公司偏休闲商务，干净整洁
- ☐ **设备**：笔记本电脑充满电（可能需要现场编码）
- ☐ **时间**：提前15-20分钟到达（或提前5分钟进入视频会议）
- ☐ **环境**：视频面试确保安静、光线好、背景整洁
- ☐ **网络**：有线优先，备热点
- ☐ **白板/共享屏幕工具**：如远程面试，确保画图工具会用
- ☐ **项目经历**：准备2-3个深度参与的项目详细描述（STAR方法：Situation/Task/Action/Result）
- ☐ **自我介绍**：准备1-2分钟简洁版（教育+经历+优势+求职动机）
- ☐ **基础题过一遍**：本文档TOP30 + 关键数字表

10.2 常犯错误提醒

错误	纠正
"我不太清楚，好像..." → 模糊回答	诚实说不知道，但展示推理思路
回答太短（一句话）	用结构化方式展开，展示思考过程
回答太长（跑题）	先给核心答案，再问"需要我深入解释吗"
不沟通直接写代码	先问清楚需求/边界条件，确认再码
写完代码不测试	带例子手动走一遍，说"我用一个例子验证"
沉默冷场	遇到难题说出你的思考过程，面试官看的是分析能力
面试结束无问题	至少准备3个有深度的问题(见第九部分)
过度自贬/自夸	客观描述即可——事实自己说话

10.3 最后一句话

面试不是考试，是双向选择——
他们选你，你也选他们。
自信 + 真诚 + 有准备 = 90%的成功。

祝面试顺利！记住：你已经做了充分的准备。深呼吸，相信自己！

补充：STL算法与常见陷阱速查

STL常用算法速记

算法	用途	复杂度	前提
sort	排序	$O(N \log N)$	RandomAccessIterator
stable_sort	稳定排序	$O(N \log N)$	归并排序
partial_sort	部分排序(TOP K)	$O(N \log K)$	前K个有序
nth_element	第N个元素归位	$O(N)$	类似快排分区
binary_search	二分查找	$O(\log N)$	有序
lower_bound	第一个 $\geq$ value	$O(\log N)$	有序
upper_bound	第一个 $>$ value	$O(\log N)$	有序
equal_range	[lower, upper)	$O(\log N)$	有序
find / find_if	线性查找	$O(N)$	无序可用
remove / remove_if	移除非移除元素	$O(N)$	配合erase
unique	去重	$O(N)$	通常先sort
transform	逐元素转换	$O(N)$	类似map
for_each	逐元素执行	$O(N)$	可修改元素
accumulate	累加	$O(N)$	numeric头
count / count_if	计数	$O(N)$	—

常见STL陷阱

- **vector 不是容器** — 不返回bool&，用deque或vector替代
- **遍历中erase** — 用 `it = c.erase(it)` (C++11起erase返回下一个有效迭代器) 或 `erase(remove_if(...), end())`
- **sort的比较函数** — 必须满足严格弱序(irreflexive/transitive/trichotomy)
- **reserve vs resize** — reserve只分配capacity不变size；resize同时改变size
- **list::size() O(1) 还是 O(n)** — C++11起规定O(1) (之前SGI STL O(n))
- **unordered_map rehash** — 插入时可能触发，迭代器全部失效
- **空的vector的data()** — C++11起data()可返回非nullptr，v[0]是UB

补充：模板元编程速查

常用type_traits

```
// 类型查询
is_same_v<T, U>, is_integral_v<T>, is_floating_point_v<T>
is_pointer_v<T>, is_reference_v<T>, is_class_v<T>, is_enum_v<T>
is_base_of_v<Base, Derived>, is_convertible_v<From, To>
is_constructible_v<T, Args...>, is_move_constructible_v<T>
is_trivially_copyable_v<T> // 可安全memcpy
is_pod_v<T> // C++20废弃, 改用is_trivial_v + is_standard_layout_v

// 类型转换
remove_const_t<T>, remove_reference_t<T>, remove_pointer_t<T>
decay_t<T>, add_pointer_t<T>, make_signed_t<T>
enable_if_t<B, T>, conditional_t<B, T, F>
common_type_t<Ts...>, underlying_type_t<E>

// 辅助: C++17起用 _v 替代 ::value, _t 替代 ::type
```

SFINAE / enable_if 常见形式

```
// 方式1: 返回类型启用
template<typename T>
enable_if_t<is_integral_v<T>, T> foo(T t) { return t * 2; }

// 方式2: 额外模板参数启用
template<typename T, enable_if_t<is_integral_v<T>, int> = 0>
T bar(T t) { return t * 3; }

// 方式3 (C++20): concepts替代
template<typename T> requires integral<T>
T baz(T t) { return t * 4; }
```

补充：系统性能分析速查

perf / strace / gdb 常用命令

```
# perf: 采样分析CPU
perf record -g ./program          # 记录调用图
perf report                       # 查看报告
perf stat ./program               # 统计信息(IPC/cache-miss等)

# strace: 追踪系统调用
strace -c ./program               # 统计系统调用次数+时间
strace -p PID                     # 附加到运行中进程
strace -e trace=network ./prog    # 只显示网络系统调用

# gdb: 调试
```

```
gdb ./program core                # 分析core dump
bt / bt full                      # 调用栈 / 含局部变量
thread apply all bt              # 所有线程调用栈
info threads                     # 线程列表
break file.cpp:123               # 设断点
p variable                       # 打印变量
watch variable                   # 监控变量变化

# 内存诊断
valgrind --leak-check=full ./prog # 内存泄漏检查
valgrind --tool=helgrind ./prog   # 数据竞争检测
g++ -fsanitize=address -g prog.cpp # AddressSanitizer编译
```

top / iostat / vmstat 指标速读

指标	含义	正常范围
load average	运行+等待CPU的任务	< CPU核数
%user	用户态CPU占用	取决于应用
%sys	内核态CPU占用	通常<30%
%iowait	等I/O的CPU时间	<5% (过高=磁盘瓶颈)
%idle	空闲率	有值即可
si/so(vmstat)	swap换入/换出	应为0(非0=内存不足)
iowait高	磁盘是瓶颈	改用SSD或优化I/O

补充：编译原理速查

编译四阶段

```
源文件(.cpp)
  | [预处理] g++ -E
  v 头文件展开、宏替换、条件编译...
预处理文件(.i)
  | [编译] g++ -S
  v 词法分析、语法分析、语义分析、生成汇编
汇编文件(.s)
  | [汇编] g++ -c
  v 转成机器码
目标文件(.o)
  | [链接] g++
  v 符号解析、重定位、合并段
可执行文件(.out/.exe)
```

Name Mangling / ABI

- C++函数签名含函数名+参数类型在编译后变为唯一符号
- `void foo(int)` 在 g++ 可能变为 `_Z3fooi`
- `extern "C"` 禁止此编码 → C/C++混合编程
- ABI兼容性: 不同编译器/版本、不同编译选项(O0/O2) 可能不兼容
- 常见ABI: Itanium C++ ABI (g++/clang++ on Linux)

链接错误速查

错误	原因	解决
undefined reference	函数声明了未定义 / 库未链接	补足定义或 -llib
multiple definition	头文件有非inline定义	移到.cpp或加inline
typeinfo / vtable error	第一个非内联虚函数未定义	确保至少一个虚函数有定义
duplicate symbol	两个.cpp定义了同名的非static函数/变量	加static或namespace或inline

补充：代码规范速查

命名规则

类型	风格	例子
类名	PascalCase	<code>class ThreadPool</code>
函数名	camelCase / snake_case	<code>getResult()</code> / <code>get_result()</code>
变量名	snake_case	<code>int thread_count</code>
常量	kPascalCase / UPPER_CASE	<code>kMaxSize</code> / <code>MAX_SIZE</code>
成员变量	尾部下划线	<code>int count_;</code>
命名空间	snake_case	<code>namespace my_lib</code>
宏	UPPER_CASE	<code>#define MAX_BUFFER 1024</code>

最重要：团队一致性 > 个人偏好。

注释原则

1. 代码即文档 — 好的命名>注释。 `int d // days` → 直接命名 `int days`
2. 注释 WHY 非 WHAT — 代码已说做什么，注释应说为什么这样做
3. TODO/FIXME/HACK — 标注临时代码
4. 不注释掉的代码 — 删掉或从git历史找（不要留下大段注释代码）

补充：GoF 23种设计模式分类速查表

创建型(Creational)

模式	核心思想
单例(Singleton)	全局唯一实例
工厂方法(Factory Method)	子类决定创建什么
抽象工厂(Abstract Factory)	创建相关对象族
建造者(Builder)	分步创建复杂对象
原型(Prototype)	克隆对象

结构型(Structural)

模式	核心思想
适配器(Adapter)	转接口
桥接(Bridge)	分离抽象与实现
组合(Composite)	部分-整体层次
装饰器(Decorator)	动态加功能
外观(Facade)	简化接口
享元(Flyweight)	共享细粒度对象
代理(Proxy)	控制访问

行为型(Behavioral)

模式	核心思想
责任链(Chain of Resp)	传递请求直到被处理
命令(Command)	请求封装为对象
解释器(Interpreter)	定义语法解释器
迭代器(Iterator)	顺序访问集合元素
中介者(Mediator)	对象交互中介
备忘录(Memento)	快照/回滚
观察者(Observer)	一对多通知
状态(State)	状态驱动行为

模式	核心理念
策略(Strategy)	算法族可互换
模板方法(Template Method)	父定义骨架，子实现步骤
访问者(Visitor)	操作与结构分离

C++ STL中的设计模式

- **迭代器模式** — STL迭代器（最完善的实现）
- **适配器模式** — stack/queue/priority_queue（容器适配器）
- **策略模式** — std::sort接收比较器对象
- **命令模式** — std::function/std::packaged_task
- **代理模式** — std::shared_ptr/std::weak_ptr（智能指针作为裸指针的代理）
- **原型模式** — virtual clone()方法

补充：Linux系统目录速查

目录	用途
/proc	内核/进程虚拟文件系统
/sys	内核/设备/驱动信息
/dev	设备文件
/etc	配置文件
/var/log	日志文件
/tmp	临时文件（可能定期清理）
/usr/lib	库文件
/usr/include	头文件
/opt	第三方软件

补充：内存对齐速查

```
// 对齐规则：成员偏移量是自身大小的整数倍，最终大小是最大对齐值的整数倍
struct A { char c; int i; };           // sizeof=8 (c占1+padding3, i占4)
struct B { int i; char c; };           // sizeof=8 (i占4, c占1+padding3至对齐边界)
struct C { char c; double d; };        // sizeof=16 (c占1+padding7, d占8)

// alignas 指定对齐
struct alignas(64) Aligned { int x; }; // 整个结构体64字节对齐

// pragma pack: 改变默认对齐 (=破坏性能，谨慎使用)
```

补充：线程数量选择指南

任务类型	推荐线程数	原因
CPU密集（纯计算）	CPU核数	超过核数只能切换，无助于加速
IO密集	2-4x CPU核数	IO等待时CPU空闲，可运行其他线程
混合型	动态调整	观察CPU利用率和IO等待时间调优

公式参考（IO密集）：线程数 = CPU核数 * (1 + 等待时间/计算时间)

全文完。 这份速记已加载就绪——深呼吸，放松，相信自己。祝你今天面试顺利！